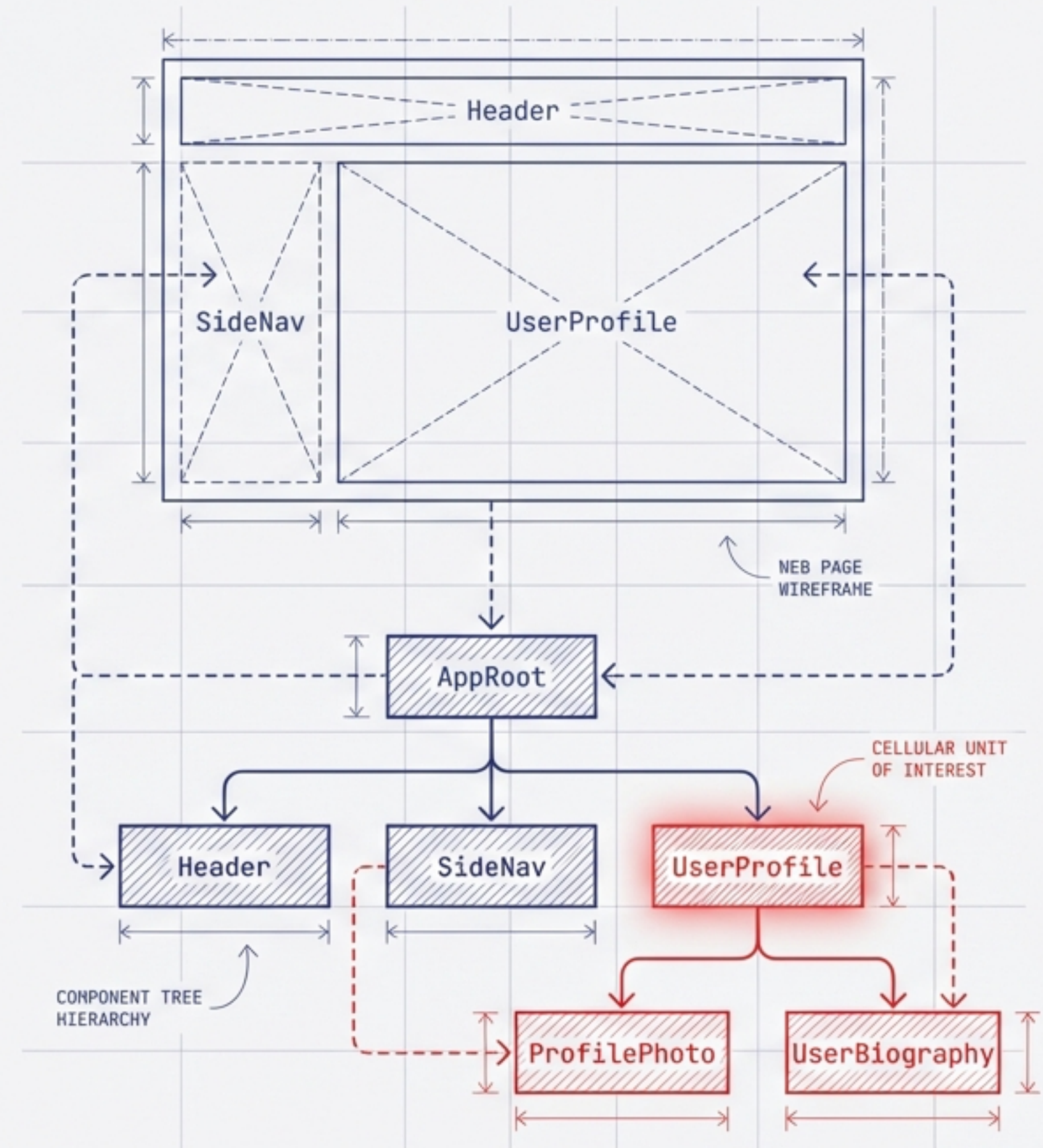


The Fundamental Cellular Unit

Angular applications are not monolithic; they are trees of components. Each component acts as a cellular unit, encapsulating its own logic and view.

- **Structure:** Separates code into manageable parts.
- **Maintainability:** Isolates concerns for easier growth.
- **Composition:** Complex interfaces built from reusable blocks.



Anatomy of a Component Definition

Dissecting the four critical organs of the component class.

The Decorator:

Stores configuration and metadata.

The Template:

Defines the HTML structure to render.

The Selector:

Defines the custom HTML tag (e.g., `<user-profile>`).

The Class:

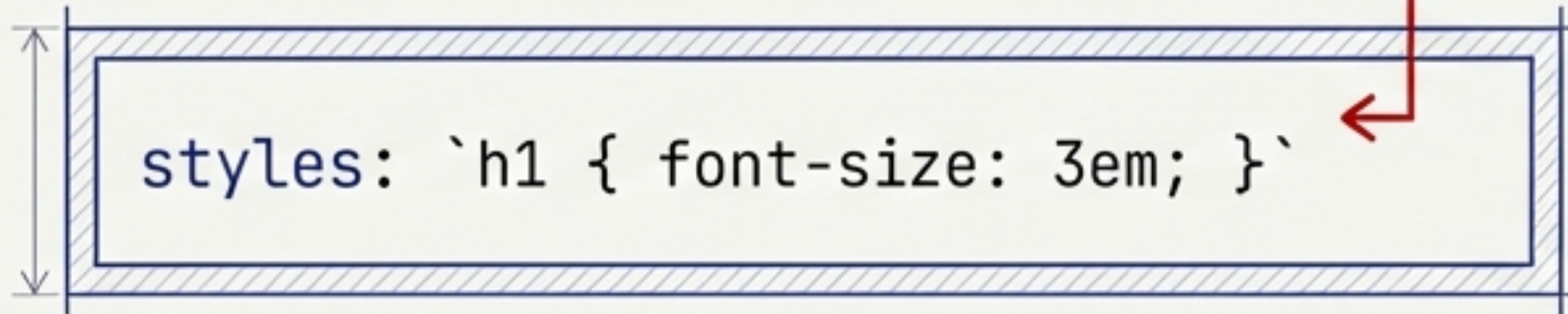
TypeScript containing state and behavior.

```
@Component({  
  selector: 'user-profile',  
  template: `<h1>User profile</h1>  
    <p>Active user</p>`,  
})  
export class UserProfile {  
  
  /* Logic goes here */  
}
```


Encapsulation and File Separation

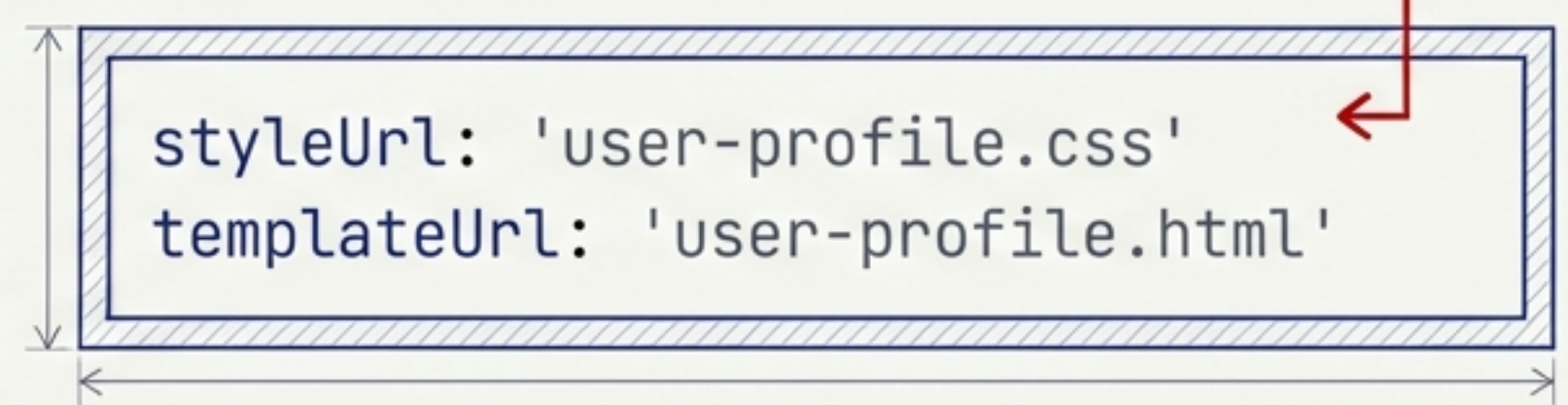
Inline - Simple Organism

Use for simple, self-contained components.



External - Complex Organism

Use for robust components requiring clarity.



Style Encapsulation



Encapsulation ensures styles defined in a component do not leak to the rest of the application.

Composition: Building the Organism

Assembling components to create complex structures.

```
import { ProfilePhoto } from 'profile-photo.ts'; // [1]

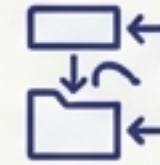
@Component({
  selector: 'user-profile',
  imports: [ProfilePhoto], // [2]
  template: `
    <h1>User profile</h1>
    <profile-photo /> <!-- [3]
  `,
})
export class UserProfile {}
```



[1] Import (TypeScript)



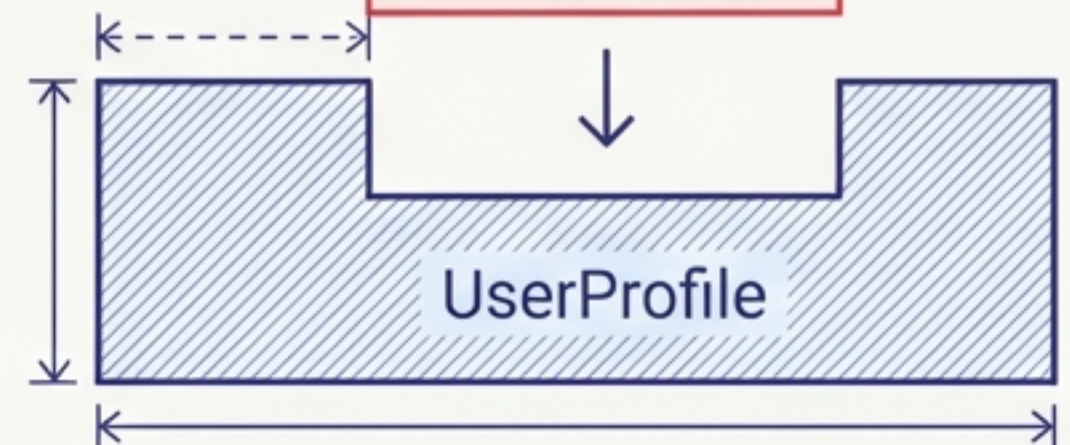
[2] Register (Decorator)



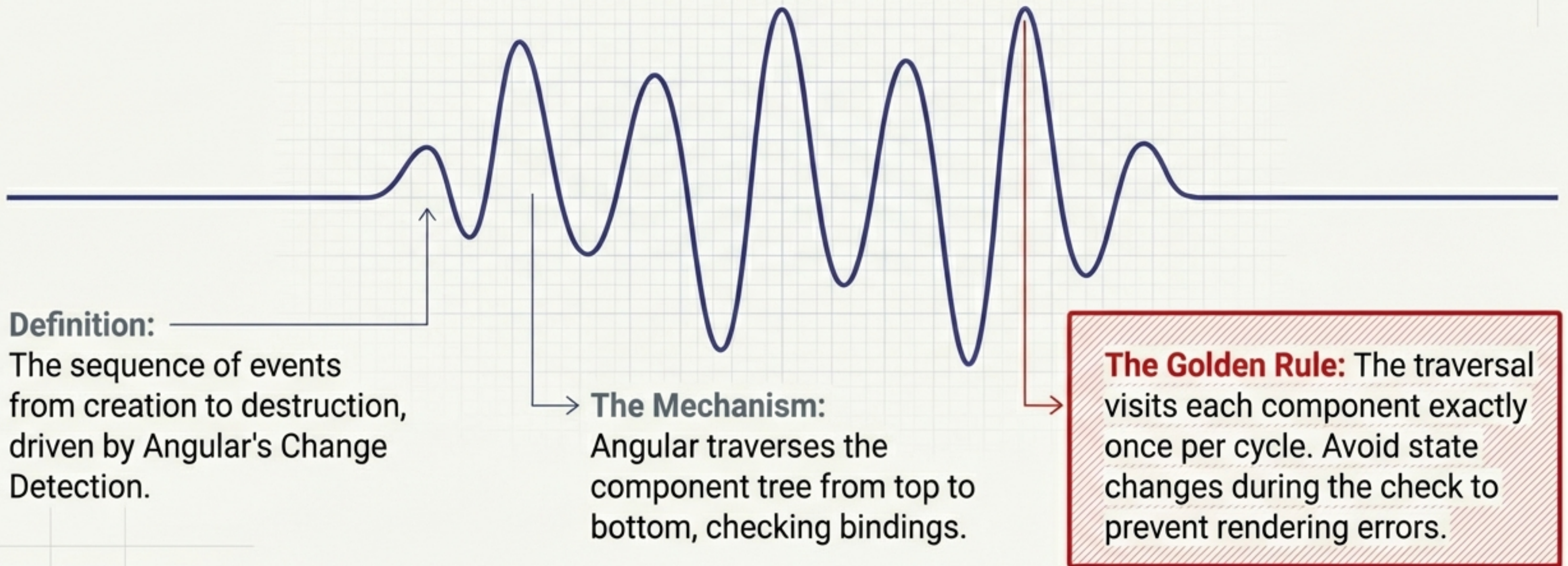
[3] Embed (HTML)



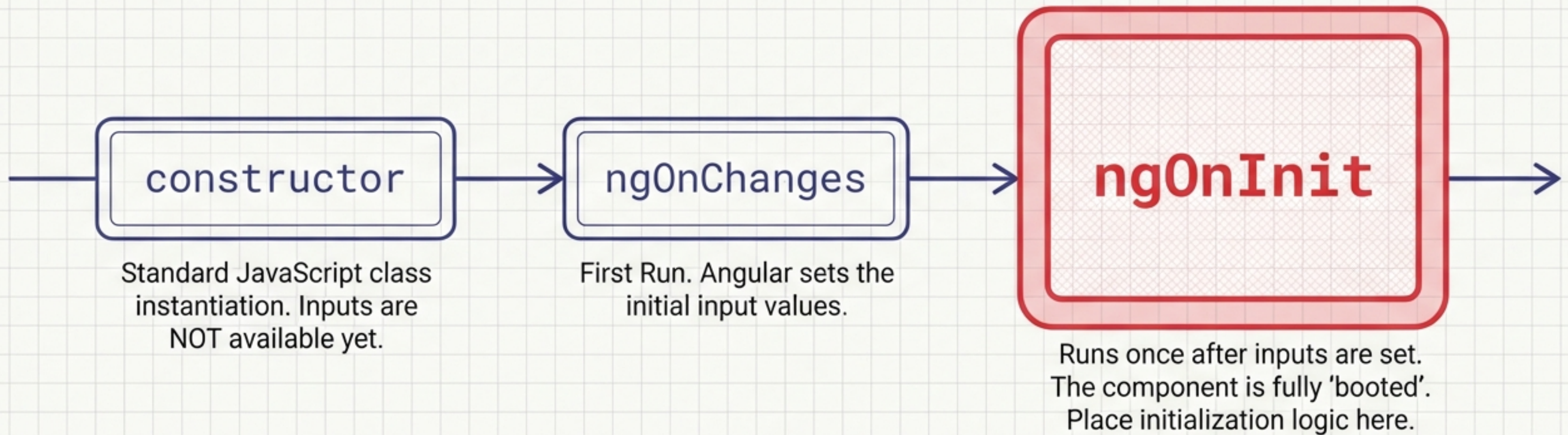
ProfilePhoto



The Pulse of the Application: Component Lifecycle

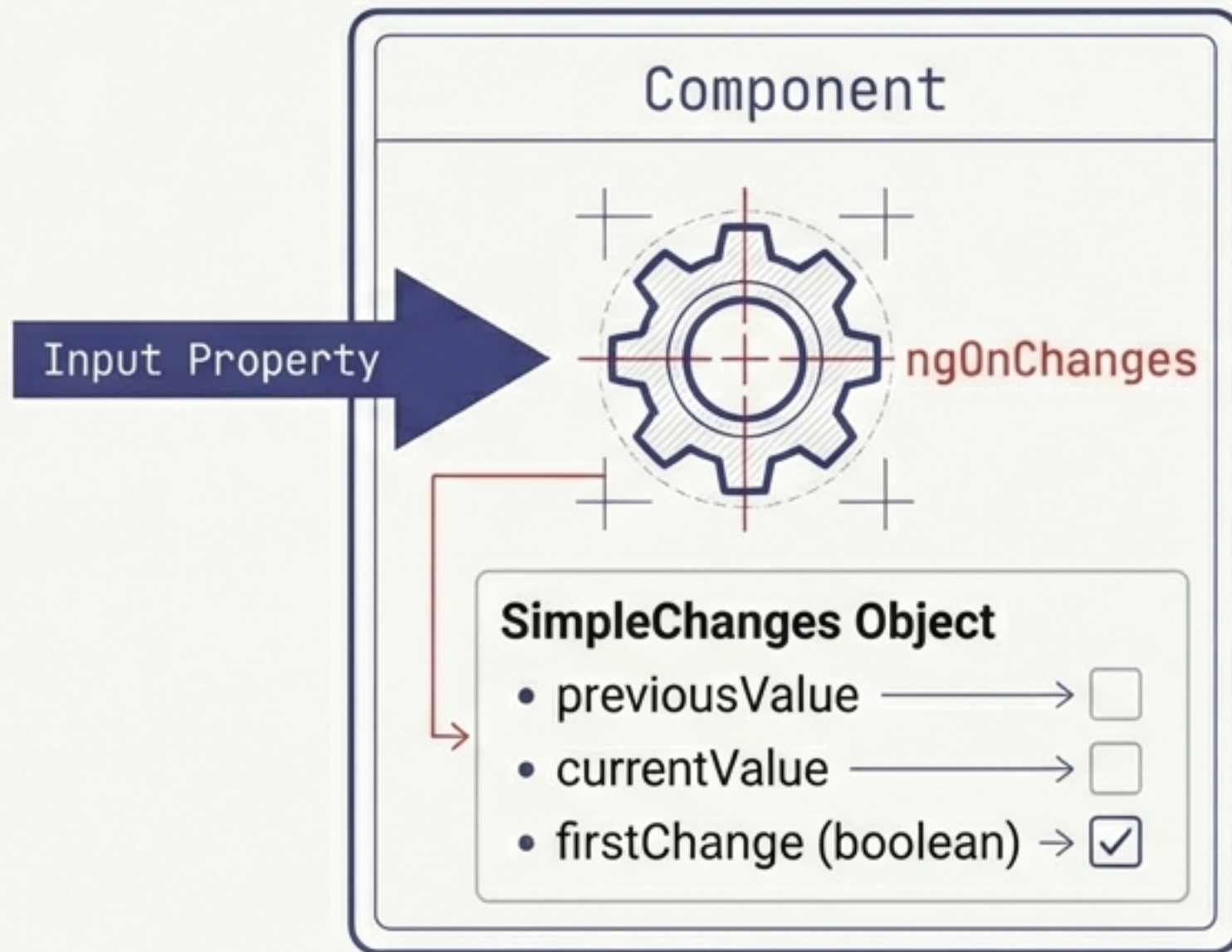


Phase 1: Creation and Initialization



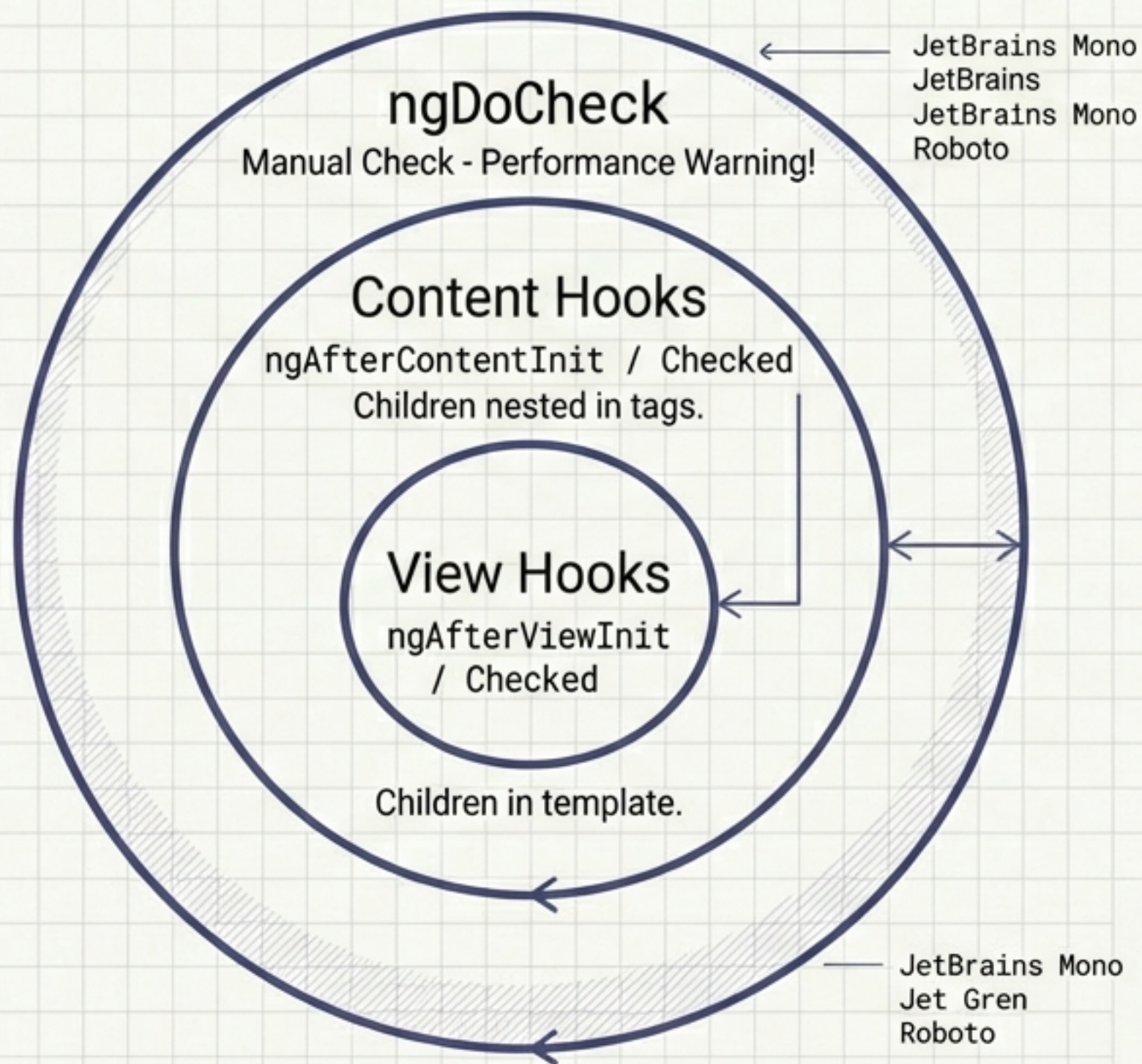
Phase 2: Reacting to Input Changes

Monitoring the `ngOnChanges` hook.



```
ngOnChanges(changes: SimpleChanges<UserProfile>) {  
  if (changes.name) {  
    console.log(`Previous: ${changes.name.previousValue}`);  
    console.log(`Current: ${changes.name.currentValue}`);  
  }  
}
```


The Check Cycle: Content and View



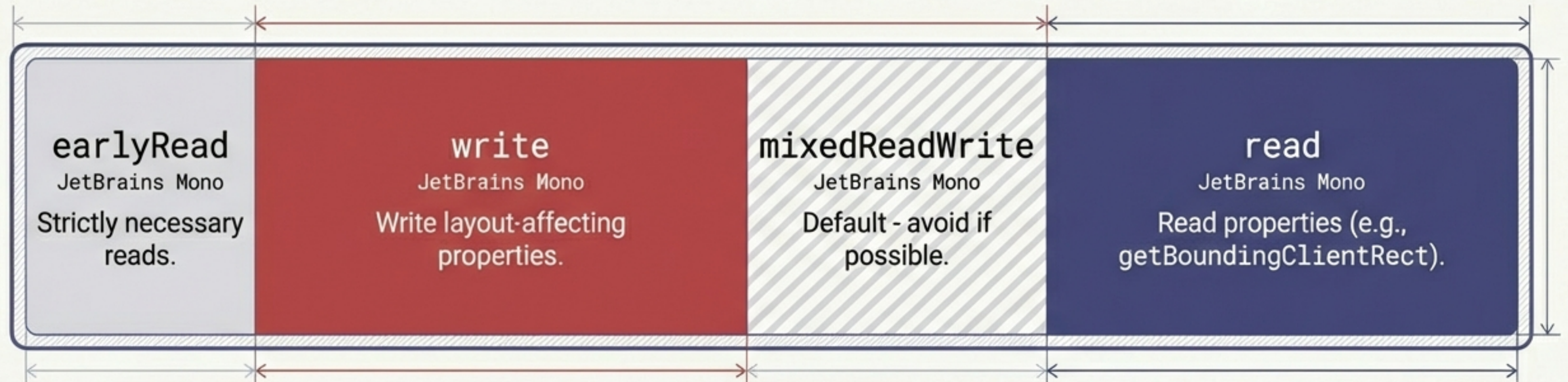
Critical Error Warning:

Changing state in 'After' hooks
(*Content/Checked* or *View/Checked*) causes
ExpressionChangedAfterItHasBeenCheckedError.

The rendering pass is complete; do not
attempt to paint again.

Modern Rendering Callbacks

Hooking into the DOM paint with `afterNextRender`.



Usage: Standalone functions (not class methods).
Must be called in an injection context (e.g., constructor).

Phase 3: Destruction and Cleanup

Legacy: Class Method

```
onDestroy() {  
  // Cleanup logic  
}
```

Runs once before component removal (e.g., @if hidden or navigation).

Modern: Dependency Injection

```
inject(DestroyRef).onDestroy(() => {  
  // Cleanup logic  
});
```

Flexible. Can be used in services or outside the class.

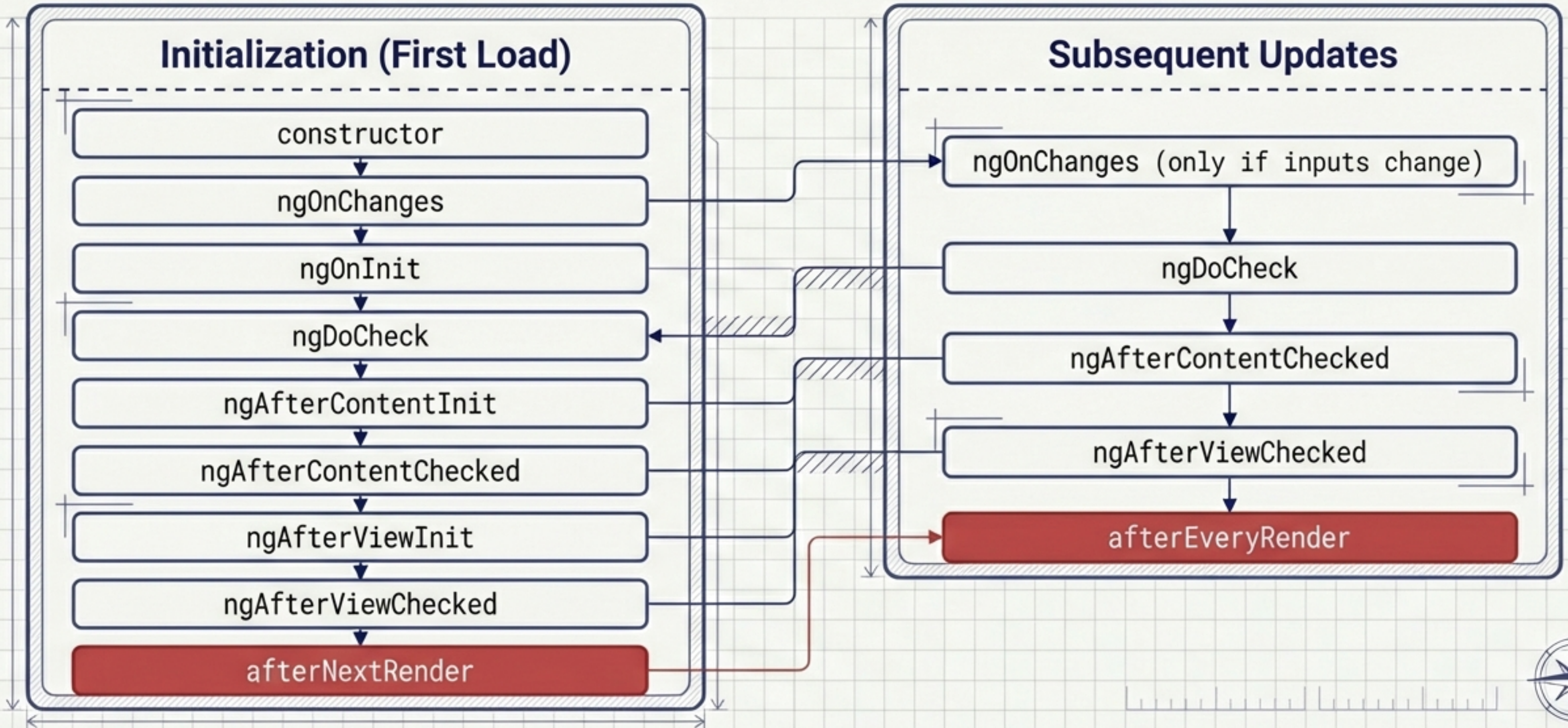
Comparison of
Legacy vs
Modern patterns.



Safety Check: Use `destroyRef.destroyed` to check if a view exists before running async updates.



Visualising Execution Order



Interfaces and Safety Patterns

Typing for Safety: Always implement lifecycle interfaces to catch typos and ensure signature correctness.

```
export class UserProfile implements OnInit, OnDestroy {  
  ngOnInit() { ... }  
  
  ngOnDestroy() { ... }  
}
```



Directives Warning: There is no guaranteed execution order between a component's hooks and the hooks of directives on the same element.

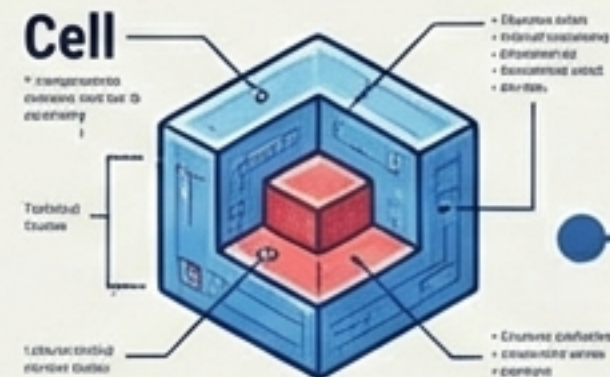


Summary of Key Methods

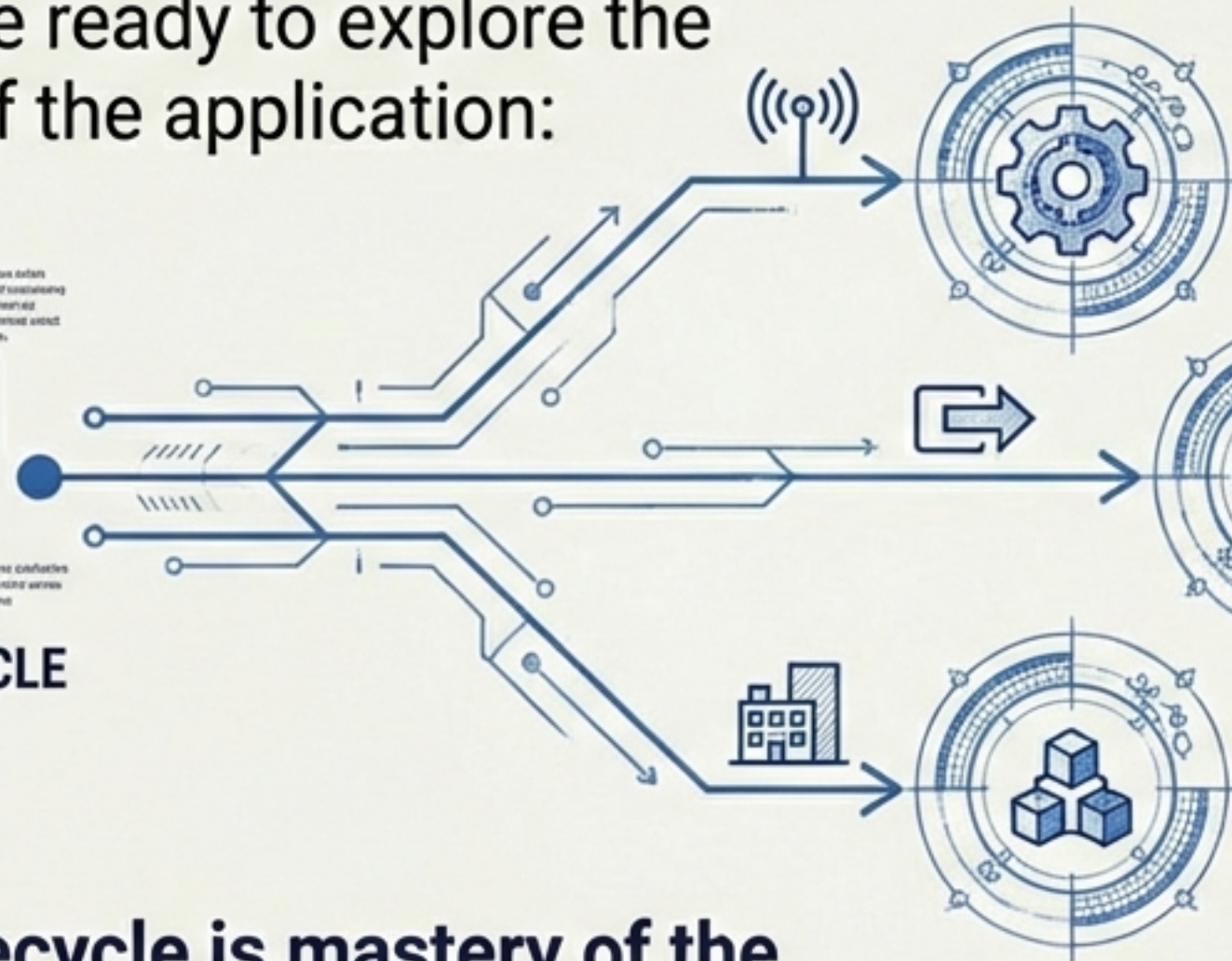
Category	Methods	Purpose
Creation	constructor, <code>ngOnInit</code>	Class instantiation & Logic requiring inputs.
Updates	<code>ngOnChanges</code> , <code>ngDoCheck</code>	Reacting to Input changes & Manual change detection.
Rendering	<code>afterNextRender</code> , <code>afterEveryRender</code>	One-time or Recurring DOM access.
Cleanup	<code>ngOnDestroy</code> , <code>DestroyRef</code>	Teardown and resource release.

Beyond the Anatomy

We have defined the static structure and dynamic lifespan of the Angular component. With this foundation, you are ready to explore the nervous system of the application:



COMPONENT LIFECYCLE



1. Reactivity with Signals

2. Interaction via Forms

**3. Modular Architecture
with Dependency Injection**

Mastery of the lifecycle is mastery of the application's performance and stability.

